

1. Line of Code (LOC) of 33200. If a team produces an average productivity of 620 LOC per month at the burdened labour rate of \$8000 per month calculate i) Cost per line of code ii) Total cost of the software iii) Estimation in person-months

- **Total LOC (Lines of Code) = 33,200**
- **Productivity = 620 LOC/month**
- **Labour cost per month = \$8,000**

i) Cost per line of code

To calculate cost per LOC:

$$\text{Cost per LOC} = \frac{\text{Labour cost per month}}{\text{Productivity (LOC/month)}} = \frac{8000}{620}$$

$$\text{Cost per LOC} \approx \$12.90$$

ii) Estimation in person-months

To calculate how many person-months are needed:

$$\text{Person-months} = \frac{\text{Total LOC}}{\text{Productivity}} = \frac{33200}{620}$$

$$\text{Person-months} \approx 53.55$$

iii) Total cost of the software

Now we calculate the total cost:

$$\text{Total cost} = \text{Person-months} \times \text{Labour cost per month} = 53.55 \times 8000$$

$$\text{Total cost} \approx \$428,400$$

Final Answers:

1. **Cost per LOC = \$12.90**
2. **Estimation in person-months = 53.55**
3. **Total cost of the software = \$428,400**

2. A team is tasked with developing a mobile application estimated to require 45,000 lines of code (LOC). The team has an average productivity of 750 LOC per month per developer, and each developer costs \$7,500 per month. Questions: a) Estimate the number of person-months required for the project. b) Calculate the cost per LOC. c) Determine the total development cost. d) If the project deadline is 10 months, how many developers are needed to meet it?

Given:

- Estimated LOC = 45,000
- Productivity = 750 LOC/month/developer
- Cost per developer = \$7,500/month
- Deadline = 10 months

a) Estimate the number of person-months required for the project

Formula:

$$\text{Person-Months} = \frac{\text{Total LOC}}{\text{LOC per month per developer}} = \frac{45,000}{750} = 60 \text{ person-months}$$

✓ Answer: 60 person-months

b) Calculate the cost per LOC

Formula:

$$\text{Cost per LOC} = \frac{\text{Cost per month per developer}}{\text{LOC per month}} = \frac{7,500}{750} = 10$$

✓ Answer: \$10 per LOC

c) Determine the total development cost

Formula:

$$\text{Total Cost} = \text{Person-Months} \times \text{Cost per Month} = 60 \times 7,500 = 450,000$$

✓ Answer: \$450,000

d) If the project deadline is 10 months, how many developers are needed to meet it?

Formula:

$$\text{Developers Needed} = \frac{\text{Person-Months}}{\text{Project Duration in Months}} = \frac{60}{10} = 6$$

✓ Answer: 6 developers

Function Point Analysis (FPA) is a standardized method for measuring the size and complexity of a software application based on its functionality from the user's perspective. It's widely used in project estimation to predict effort, cost, and schedule.

1. Why Use Function Points?

- **Language- and Technology-Independent:** FP counts derive from the user's requirements, not the underlying code or platform.
 - **Early Estimation:** You can estimate before any code is written, using requirements or high-level designs.
 - **Benchmarking & Productivity:** By tracking effort per function point, organizations can benchmark teams and improve productivity.
 - **Scope Control:** FP counts make it easier to compare proposed versus delivered functionality.
-

2. Core Components of Function Points

Function Point Analysis breaks an application's functionality into five types of components:

Component	Definition
External Inputs (EI)	User-originated inputs that update internal data (e.g., form submissions).
External Outputs (EO)	Reports or messages sent to the user or other systems, with derived data (e.g., invoices).
External Inquiries (EQ)	Interactive queries that retrieve data without updates (e.g., search screens).
Internal Logical Files (ILF)	Collections of logically related data maintained by the system (e.g., customer table).
External Interface Files (EIF)	Data used by the system but maintained by another (e.g., a shared lookup table).

3. Step-by-Step Calculation

3.1 Count Unadjusted Function Points (UFP)

1. **Identify** all EIs, EOs, EQs, ILFs, and EIFs in the system.
2. **Classify** each component's complexity (Low, Average, High) based on:
 - Number of Data Element Types (DETs): unique fields.
 - Number of File Types Referenced (FTRs): files read/updated
3. **Apply Weights** from the standard table:

Component	Low	Avg.	High
EI	3 FP	4 FP	6 FP
EO	4 FP	5 FP	7 FP
EQ	3 FP	4 FP	6 FP
ILF	7 FP	10 FP	15 FP
EIF	5 FP	7 FP	10 FP

EI	3 FP	4 FP	6 FP
EO	4 FP	5 FP	7 FP
EQ	3 FP	4 FP	6 FP
ILF	7 FP	10 FP	15 FP
EIF	5 FP	7 FP	10 FP

4. **Sum** all weighted counts to get the **Unadjusted Function Point (UFP)** total.

3.2 Adjust for Technical Complexity

To account for environmental and technical factors, FPA uses a **Value Adjustment Factor (VAF)** based on 14 general system characteristics (GSCs), such as performance, reusability, and data communications.

1. **Rate** each GSC on a scale from 0 (no influence) to 5 (strong influence).
2. **Compute** the Total Degree of Influence (TDI): sum of all 14 ratings.
3. **Calculate** VAF:

$$VAF = 0.65 + (0.01 \times TDI)$$

1. **Adjusted Function Points (AFP):**

$$AFP = UFP \times VAF$$

4. Using Function Points for Estimation

Once you have AFP:

1. **Productivity Rate:** Determine your team's average effort per FP (e.g., person-hours/FP).
2. **Effort Estimate:**

$$\text{Total Effort} = AFP \times \text{Hours per FP}$$

3. **Schedule & Cost:** Break effort into tasks, apply resource rates to get cost.

5. Benefits & Considerations

- **Pros:** Early visibility, objective sizing, supports benchmarking.
- **Cons:** Requires experienced analysts for accurate counts; initial setup of complexity rules and historical productivity rates.

Example Snapshot

Component	Count	Complexity	Weight	Total FP
EI	12	Avg.	4	48
EO	8	High	7	56
EQ	5	Low	3	15
ILF	6	Avg.	10	60
EIF	3	Low	5	15
UFP				194

If TDI = 30, then

$$VAF = 0.65 + (0.01 \times 30) = 0.95$$

$$AFP = 194 \times 0.95 \approx 184.3 \text{ FP}$$

If historical productivity is 20 person-hours/FP:

$$\text{Total Effort} \approx 184.3 \times 20 = 3,686 \text{ person-hours}$$

Two modules are being reviewed.

- Module A: All functions perform steps in processing a loan application.
- Module B: Contains unrelated functions grouped together just because they're all file-related (readFile, savePDF, logToFile).

Questions:

- a) Which module shows **functional cohesion**? Which one shows **logical cohesion**?
- b) Why is **functional cohesion** generally preferred?
- c) How could Module B be reorganized for better cohesion?

Answer:

- **Module A** shows **functional cohesion**
All its functions contribute to a single, well-defined task: processing a loan application.
- **Module B** shows **logical cohesion**
The functions are grouped because they're related to files, but they serve different, unrelated purposes (reading, saving, logging).

Why is functional cohesion generally preferred?

Answer:

Functional cohesion is preferred because:

- **Easier to maintain** – All functions in the module work together toward a single purpose.
- **Better readability** – It's clear what the module is responsible for.
- **More reusable** – Modules with a clear purpose can often be reused elsewhere.
- **Fewer side effects** – Since the module does one thing, changes are localized and less likely to affect unrelated parts of the system.

Logical cohesion, in contrast, can make modules harder to maintain and understand because:

- Functions are unrelated except by category (e.g., all file-related), which leads to bloated, catch-all utilities.
- Changes to one part of the module may unintentionally affect other, unrelated parts.